Midterm Project
CMPSC 448

Evan Grebner, Matthew Steele, Raouf Halabi

October 11, 2026

NOTE (Regarding Group Leader): We did not designate one person as a group leader as we felt and would rather have the three points dissipated between the three of us (so one point to each person) instead of just one person getting all three points, if possible. If that isn't possible, we can delegate it to Evan Grebner.

1 Introduction and Research Questions

Different LLMs have recognizable habits: some write long structured answers with headers and bold text, others write short plain paragraphs. We test how far those habits can be used to identify the author model. We treat this as a classification problem: given a response, the model has to predict which of five LLM families (Claude, Gemini, GPT, Llama, or Mistral) wrote it. We studied four research questions:

- **RQ1.** Can we identify which LLM generated a response? Use LLM_output as input, conduct classification to detect which LLM generated it. Report the performance of your CNN and RNN performance
- **RQ2.** Does the user's prompt help identify the LLM? Compare classifiers using: Input only; Output only; Input + Output. Questions you may consider: i) Can the LLM be predicted from the user's prompt even before seeing its response? ii) Does including the input improve classification? iii) If input-only classification works surprisingly well, why might this happen?
- **RQ3.** (EXTRA CREDIT) Investigate whether a classifier trained on one type of task can identify LLMs on another type of task. For example: Training on problems like coding + mathematics + factual questions, but Testing on writing or Training on general question answering but Testing on coding. Questions you may consider: i) Does performance decrease on unseen tasks? ii) Are some models more consistently identifiable across domains? iii) Is the classifier learning a general LLM fingerprint or task-specific patterns?
- **RQ4.** (EXTRA CREDIT) Classification accuracy tells us whether models can be distinguished. This RQ asks why. Analyze linguistic or structural characteristics such as: response length; vocabulary; lexical diversity; formatting, etc. Questions you may consider: i) What signals does the classifier appear to use? ii) Are the signals linguistic, structural, or both? iii) Does removing certain signals substantially reduce classification performance?

2 Data

We used the `lmarena-ai/arena-human-preference-55k` dataset, a public collection of chatbot conversations from LMArena (`https://huggingface.co/datasets/lmarena-ai/arena-human-preference-55k`). It is too large for GitHub, so our code downloads it directly. In it, two anonymous chatbots were asked the same question, so each row has one prompt and two responses. Each example therefore has the three required parts: the LLM name (from the model columns), the prompt (the LLM input), and the response (the LLM output). We turn each row into two examples and keep only the first turn of the conversation. The dataset is publicly available on Hugging Face under the Apache 2.0 license.

Labels. Each response is labeled with the family of the model that wrote it (for example, `gpt-4` and `chatgpt` both become `gpt`). We keep only English responses between 50 and 6,000 characters, and we remove duplicates.

Balanced classes. We drop families with fewer than 2,000 examples (Qwen, Yi, DeepSeek), which leaves five: Claude, Gemini, GPT, Llama, and Mistral. We then sample the same number of examples from each, so a model cannot score well just by guessing the most common family. This gives 20,850 examples, split into 14,590 for training, 3,107 for validation, and 3,153 for testing. Random guessing would get 20%.

Family	Examples after cleaning
GPT	28,920
Claude	15,130
Mistral	8,317
Llama	7,918
Gemini	4,170
Qwen / Yi / DeepSeek (dropped)	1,887 / 1,364 / 747

Table 1: Examples per family before balancing. We kept the five families with at least 2,000 examples and sampled 4,170 from each.

No leakage. The same prompt never appears in both the training and test data, so a model cannot succeed by memorizing prompts.

Task types. RQ3 needs to know what kind of task each prompt is (coding, math, writing, or question answering). We assign this with simple keyword rules, for example "python" or "sql" in the prompt means *coding*. These labels are approximate.

3 Methods

Text processing. Text is split into words, punctuation marks, and line breaks. We keep the first 100 tokens of the prompt and the first 250 tokens of the response. Because truncation hides response length, which is a strong clue, we add a special token recording the full response length in bins of 50 tokens. The vocabulary (at most 30,000 tokens, each seen at least twice) is built from training data only.

Models. We compare three models:

- *TextCNN*: a neural network that scans the text for short word patterns (2 to 5 words long) and uses the strongest matches to decide.
- *BiLSTM*: a neural network that reads the text from both directions, word by word, and uses the whole sequence to decide.
- *TF-IDF baseline*: a simple, non-neural model that looks only at which words and word pairs appear. It checks whether the neural networks add anything.

Architecture details. Both networks start by turning each word into a list of 128 numbers, learned from scratch during training.

- *CNN:* Scans the text with 256 small detectors each for phrases 2, 3, 4, and 5 words long. It keeps the strongest match from each detector (1,024 values in total) and uses them to pick the answer.
- *BiLSTM:* Reads the text forward and backward. It combines the average and the strongest signal across all words (512 values in total) and uses them to pick the answer. Padding is ignored.

Both models also use a few standard safeguards: dropout (randomly switching off some units during training so the model does not memorize), a small penalty on large weights, and a cap on how far the weights can change in one step. Both were trained on a Google Colab GPU.

Training. Both neural networks use the Adam optimizer with a learning rate of 5×10^{-4} (cut in half when progress stalls) and a batch size of 64. Training runs for at most 10 epochs and stops early if validation accuracy does not improve for 6 epochs. We then keep the version of the model that scored best on the validation set. All results come from a single run with seed 42.

Setting	Value
Word vector size	128
CNN	256 filters each of width 2, 3, 4, and 5
BiLSTM	128 hidden units per direction
Dropout	0.2 on the CNN's word vectors; 0.5 before the final output (both models)
Loss	Cross-entropy
Optimizer	Adam, learning rate 5×10^{-4}, halved when validation stalls
Batch size	64
Epochs	At most 10 (stop after 6 epochs with no improvement)
Weight penalty	10^{-5}
Gradient cap	1.0
Input length	100 prompt tokens, 250 response tokens
Vocabulary	30,000 tokens
Seed	42

Table 2: Training settings for both neural networks.

RQ4 analysis. To see which writing habits matter, we measure 23 simple style features for each response, such as length, vocabulary variety, bullets and headers, and punctuation. We train a

random forest and a logistic regression on these features alone. We sort the features into five groups, then test accuracy with each group removed and with each group used alone. We also retrain the CNN on text with formatting symbols stripped out, and again with the text lowercased as well.

4 Results

4.1 RQ1: Can the response alone identify the LLM?

Model	Test accuracy	Macro-F1
TF-IDF baseline	0.621	0.618
CNN	**0.657**	**0.653**
BiLSTM	0.611	0.609

Table 3: Accuracy using the response only (chance = 0.20).

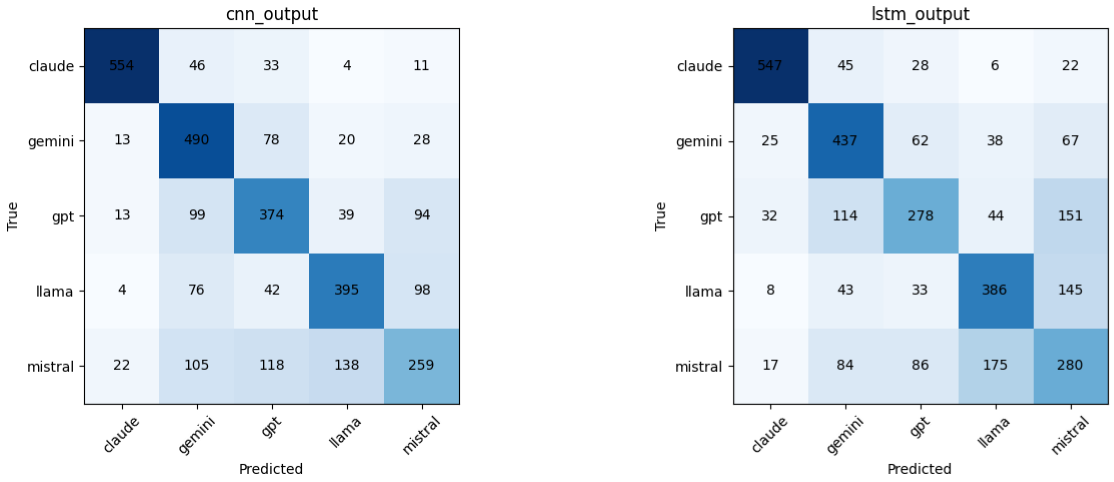


Figure 1: How often each model guessed right (the diagonal) and wrong (everything else), using the response only. Rows are the real LLM, columns are the guess.

Answer: yes. Out of 3,153 test responses, the CNN picks the right LLM for 2,072. Random guessing would get about 631. The simple TF-IDF baseline gets 1,957 right, so the CNN is ahead by 115. The BiLSTM gets 1,928, which is below the baseline. Gaps of a few dozen responses can be luck. For example, the BiLSTM is only 29 behind the baseline, which is too small to call a real difference.

Some LLMs are easier to spot than others. The CNN gets 554 of 648 Claude responses right and 490 of 629 Gemini responses right, but only 259 of 642 Mistral responses. Mistral is most often mistaken for Llama (138 times), GPT (118), and Gemini (105).

4.2 RQ2: Does the prompt help?

Input	TF-IDF baseline	CNN	BiLSTM
Response only	0.621	**0.657**	0.611
Prompt only	0.230	0.215	0.208
Prompt + response	0.608	0.644	0.616

Table 4: Test accuracy by input type (chance = 0.20). Macro-F1 values are within 0.01 of accuracy in this table.

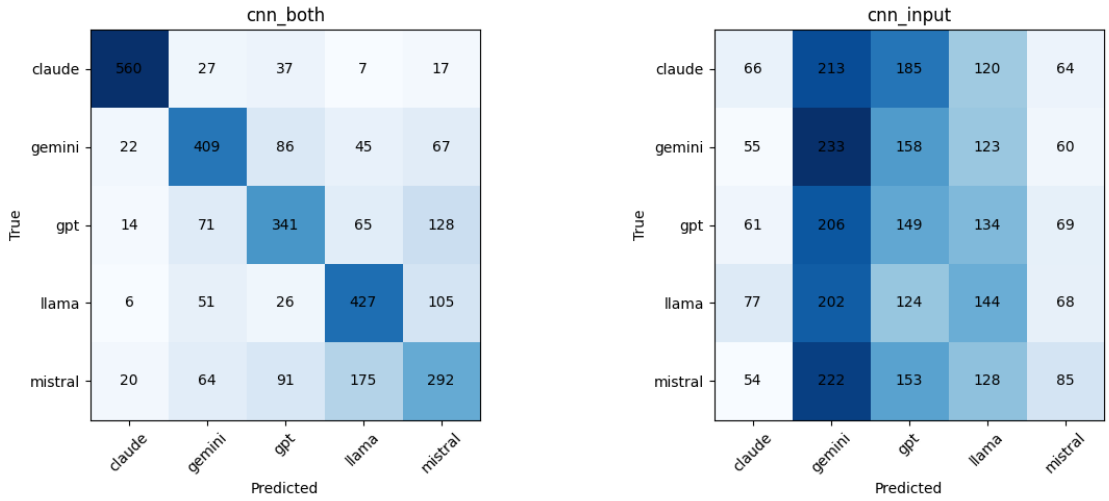


Figure 2: Left: the CNN's guesses using the prompt and response together. Right: the CNN's guesses from the prompt only. Rows are the real LLM and columns are the guess.

Answer: no. With only the prompt, the models get between 655 and 724 of 3,153 right. Random guessing would get about 631, so they are basically guessing. This makes sense, because in our dataset the user doesn't know which models they are talking to, so their prompt has nothing to do with which LLM answers. Adding the prompt to the response does not help either. The CNN gets 2,029 right with both, compared with 2,072 with the response alone.

4.3 RQ3: Do LLM fingerprints generalize across tasks?

Setup. We hid one task type from training and tested on it. We did this four times, once each for coding, math, QA, and writing. Each time, the models trained on the other three task types and were then tested on every example of the hidden one. The models see only the response, as in RQ1. Task types come from simple keyword rules (Section 2), so they are approximate. For comparison, when the models saw all four task types in training (RQ1), the CNN scored 0.657 and the BiLSTM scored 0.611.

Held-out task	CNN	vs. RQ1	BiLSTM	vs. RQ1
Coding	0.554	−10.3	0.478	−13.3
Math	0.583	−7.4	0.558	−5.3
QA	0.568	−8.9	0.506	−10.5
Writing	0.578	−7.9	0.586	−2.5
Average	0.571	−8.6	0.532	−7.9

Table 5: Test accuracy on a task type excluded from training (response only). "vs. RQ1" is the change in percentage points from the RQ1 accuracy. Macro-F1 is within 0.02 of accuracy in every row. Chance is 0.20.

Does performance drop on unseen tasks? Yes, but it stays far above chance. Averaged over the four tasks, the CNN falls from 65.7% to 57.1% and the BiLSTM from 61.1% to 53.2%, so both keep about 87% of their original accuracy. The CNN drops in all four cases (7.4 to 10.3 points). The BiLSTM drops in all four as well, but by very different amounts (2.5 to 13.3 points). Even the worst result (BiLSTM on coding, 47.8%) is more than twice chance.

Which classifier is steadier? The CNN is more stable. Its accuracy stays between 55.4% and 58.3% on every task, a spread of 2.9 points. The BiLSTM ranges from 47.8% (coding) to 58.6% (writing), a spread of 10.8 points. On writing the BiLSTM is slightly ahead of the CNN (58.6% vs. 57.8%), but this is one run and the gap is too small to call a real difference.

General fingerprint or task-specific patterns? Both. Most of the signal is general: a model that has never seen a task still gets about 87% of its usual accuracy on it, so what it learned is mostly style that carries over. The remaining loss of about 8 points shows that part of what the models learn depends on the task. This fits RQ4, where formatting was the strongest cue, since formatting habits (code blocks, bullet lists, headers) may differ from one task to another. Coding was the hardest task to transfer to for both models. A possible reason is our labeling rule: a response containing a code block is labeled coding, so the training data for the other three tasks has far fewer code blocks, and the models saw little of how each LLM formats code. We did not test this directly.

Caveats. Each result is from a single run. The held-out test sets are different sizes from the RQ1 test set, so the comparison is approximate. Gaps of a point or two (for example, between the CNN and BiLSTM on writing) should not be read as real differences. The roughly 8-point drop is consistent across all four tasks for the CNN, so we are more confident in it.

4.4 RQ4: What distinguishes the LLMs

A random forest using just the 23 style features picked the right LLM family 54.7% of the time, while logistic regression got 39.7%. The forest most likely did better because it can pick up on how features work together, and logistic regression cannot. The features the forest relied on most were vocabulary variety, response length, average word length, and how often commas and periods are used.

Feature group	Without group	Only group
Length	0.523	0.288
Vocabulary	0.520	0.240
Formatting	**0.472**	**0.380**
Punctuation	0.500	0.348
Characters	0.527	0.228
All features	0.547	

Table 6: Random-forest test accuracy when a feature group is removed or used on its own.

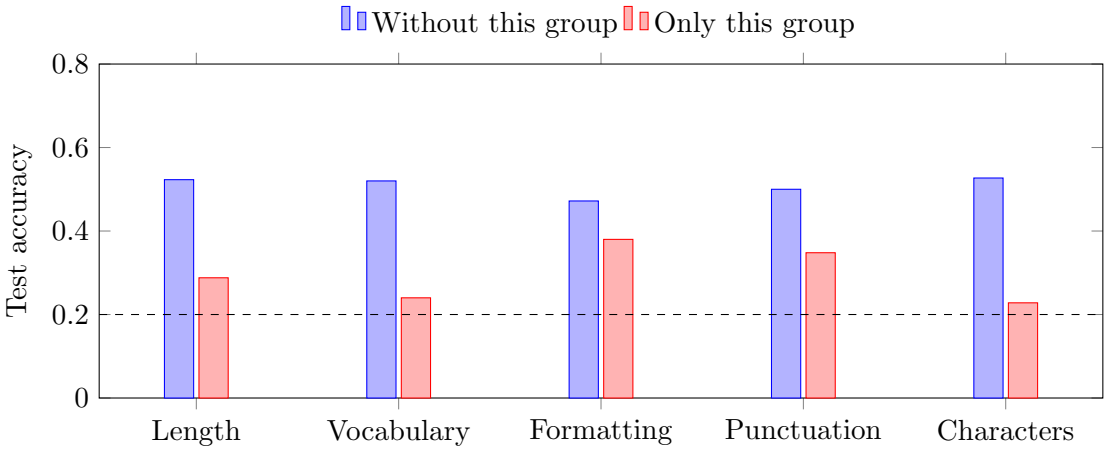


Figure 3: Accuracy when each feature group is taken out (blue) or used by itself (red). The dashed line shows random guessing (0.20).

Formatting matters most. When we removed the formatting features, accuracy dropped by 7.5 points, which was more than any other group. Formatting alone reached 38.0%, also the highest of any single group. Punctuation came in second, reaching 34.8% alone and losing 4.7 points when removed. The length, vocabulary, and character features were weak alone (23–29%) and didn't make a big difference when removed (2–3 points).

Feature importance versus ablation. The forest's importance ranking put vocabulary and length features at the top, but removing those groups barely changed accuracy. This is because they overlap with other features, so the forest can get the same information somewhere else. Formatting doesn't have a backup like that, so we choose to trust the ablation results more than the importance ranking.

Text-level check. We also retrained the CNN on text with the formatting symbols removed. Its

accuracy dropped from 65.7% to 60.8%, and stripping and lowercasing the text as well brought it to 59.2%. Formatting is worth about 5 points to the CNN, but since it still reached about 60% without it, the way each model words things matters too. So the signals are both. Structure (formatting and length) is the strongest single cue, but wording still carries real signal.

5 Discussion

Each LLM family has a detectable style, and the cheapest cues are formatting and wording. A simple TF-IDF model captures most of what the neural networks do, so the best neural model improves on it by only a few points. The result that the prompt gives no information, and that unseen task types cost 7–10 points, supports reading the response-only accuracy as a style effect and not a memorized prompt effect.

6 Limitations

- **Single run.** Every number comes from one run with one seed. Small gaps, such as BiLSTM versus baseline, are within sampling noise.
- **Approximate task labels.** Tasks come from keyword rules, so "coding" or "qa" may contain mislabeled examples, which affects RQ3.
- **Family-level labels.** Grouping models into families hides differences between versions (for example, GPT-3.5 versus GPT-4).
- **Arena data.** Responses come from one platform, with users who chose to compare models. Results may not carry over to other settings, and models and system prompts change over time.
- **Filtering.** We kept only mostly-English text and dropped small families, so the results apply to the five largest families.
- **Training length.** Many CNN runs were still improving at epoch 10, so a longer schedule might add a little accuracy.

7 Lessons and Experience

Runtime. Our first version of the code took way too long to run on our own laptops, with a full run being estimated at over an hour. We fixed this by moving to Google Colab and using its free GPU. This brought the pipeline down to about 8 minutes. This was very important to our ability to run the code and make changes quickly.

Finding usable data. Finding a dataset that we could use and followed the project requirements was harder than we were expecting. We needed the model name, the prompt, and the response for every example, from at least three LLM families. We were able to find a dataset that had all of these things, but the file was too big to upload to GitHub, so our code downloads it directly.

Avoiding leakage. We learned that how the data is split matters a lot. Because two models answered every prompt, the same prompt could end up in both training and testing. We had to make sure that each prompt only appears in one split, so the models would learn each LLM's style instead of just memorizing the prompts.

Formatting as a clue. We were surprised to see how much simple formatting gives a model away. Bullets, headers, and paragraph breaks alone got 38.0% accuracy, and removing the formatting from the text cost the CNN about 5 points.

What we would do differently. Every result in this report comes from a single run with one seed, which means that some of the small differences could just be luck. In the future, we would run the experiments multiple times with many different seeds and take the average of the results. This would make our comparisons and results more reliable.

Simple baselines matter. A simple TF-IDF model got 1,957 of 3,153 test responses right, and our best neural network (the CNN) got 2,072. The neural networks only helped a little, so we would always run a simple baseline first.

Training longer. Validation accuracy was still creeping up at epoch 10, and early stopping never triggered. With more time, we would train for more epochs.

8 Conclusion

A response alone identifies its source LLM family with about 66% accuracy in a five-way task, roughly three times chance. The prompt carries no information. The signal weakens but survives when a task type is held out, and formatting is the strongest stylistic cue.

Reproducibility

Code and results are in the course repository (`https://github.com/roof37/CMPSC448_Midterm1`), run on a Colab GPU with seed 42.